Rapport de Laboratoire 3 — SEG 3502

Angular — Composants Standalone, Formulaires Templates, Inputs/Outputs et Observables RxJS

Étudiant 1 : Anwar `N° 300254189`

Étudiant 2 : Manasse `N° 300268472`

Cours : SEG 3502 (Conception de logiciels distribués)

Date : Octobre 2026 | **Projet :** `lab3-shopping-list`

Dépôt GitHub du projet : https://github.com/Anwarito-Ibd/SEG3502-Lab3.git

1. Brève Description du Travail Réalisé

Ce troisième laboratoire approfondit l'architecture basée sur les **composants autonomes (Standalone Components)** d'**Angular (v22)**. Deux volets complémentaires ont été implémentés et articulés :

- **Livrable officiel évalué — Application de Liste de Courses (Shopping List) :**
  Conforme à 100% au wireframe officiel (page 14 du manuel), implémentée avec une architecture modulaire composée de **4 composants distincts** (dépassant le minimum requis de 2) et un service d'état :
  - `ShoppingInputComponent` (`<app-shopping-input>`) : champ de saisie réactif lié par `[(ngModel)]`, validation anti-vide, raccourci clavier Entrée (`keyup.enter`), et émission d'événement `@Output() itemAdded` .
  - `ShoppingListDisplayComponent` (`<app-shopping-list-display>`) : composant d'affichage recevant les articles via `@Input() items` et propageant la suppression via `@Output() itemDeleted` .
  - `ShoppingItemComponent` (`<app-shopping-item>`) : sous-composant individuel affichant chaque badge bleu arrondi stylisé (ex: « *5 pommes* », « *12 oeufs* », « *1 pain* ») et le bouton « *Supprimer* » associé.
  - `ShoppingListComponent` (`<app-shopping-list>`) : composant conteneur assemblant l'ensemble dans une carte bordée fidèle au design, incluant un bouton de réinitialisation.
  - `ShoppingService` : service d'état réactif centralisé basé sur un `BehaviorSubject<string[]>` de **RxJS**.
- **Tutoriel complet — Carnet d'adresses (Address Book) :**
  Réalisation de l'intégralité du tutoriel (pages 1 à 13) : modèle typé `AddressEntry` , barre d'en-tête `HeaderComponent` , liste dynamique avec surbrillance `[ngClass]` (`AddressListElement`), formulaire d'édition protégée/modifiable (`AddressView`), et service de diffusion découplé (`NotificationService`).
- **Navigation par onglets :** Le composant racine `App` fournit un basculement instantané entre l'exercice évalué (Liste de courses) et le tutoriel (Carnet d'adresses).

2. Captures d'Écran de l'Application

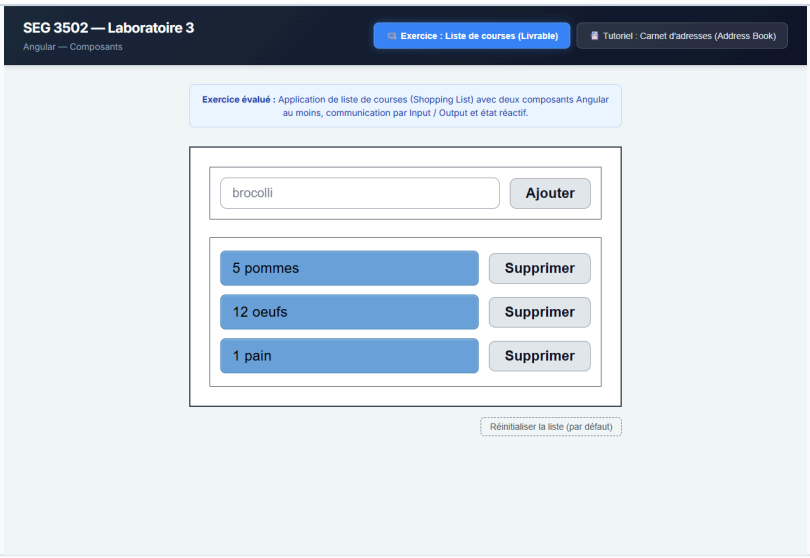


Figure 1 : Application Shopping List (Exercice évalué) — Interface reproduisant fidèlement le wireframe du laboratoire : barre d'ajout réactive, badges bleus arrondis pour les articles, et boutons de suppression individuels.

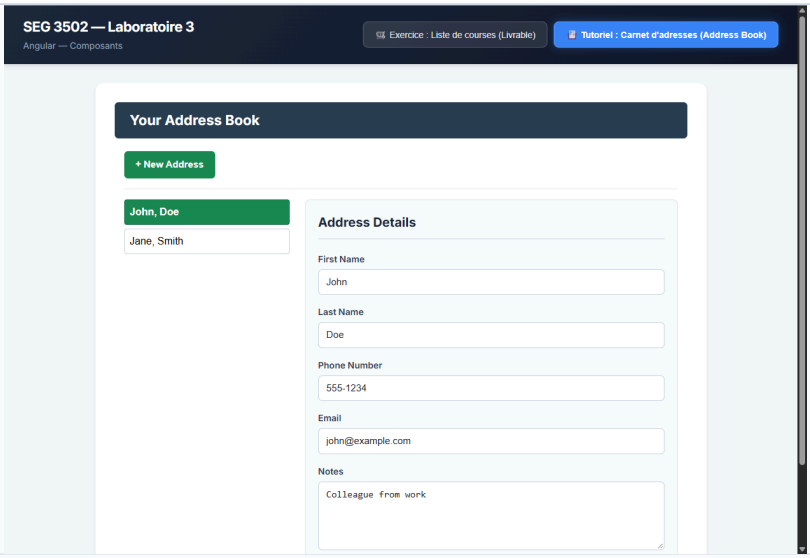


Figure 2 : Application Carnet d'adresses / Address Book (Tutoriel) — Liste de sélection à gauche avec synchronisation RxJS et formulaire d'édition désactivable/protégé à droite.

3. Validation et Suite de Tests Automatisés (Vitest)

Une suite rigoureuse de **31 tests unitaires** répartis sur **10 fichiers de spécification** valide le bon fonctionnement de tous les services et composants :

Fichier de Spécification	Composant / Service	Tests	Résultat
`shopping.service.spec.ts`	`ShoppingService` (CRUD, trim, reset)	6	Succès
`shopping-input.spec.ts`	`ShoppingInputComponent` (saisie, Output, validation)	4	Succès
`shopping-item.spec.ts`	`ShoppingItemComponent` (rendu, Output delete)	2	Succès
`shopping-list-display.spec.ts`	`ShoppingListDisplayComponent` (Inputs, propagation)	3	Succès
`shopping-list.spec.ts`	`ShoppingListComponent` (orchestration conteneur)	4	Succès
`address-entry.spec.ts`	`AddressEntry` (modèle de données typé)	2	Succès
`notification-service.spec.ts`	`NotificationService` (BehaviorSubject RxJS)	2	Succès
`address-view.spec.ts`	`AddressView` (bascule Edit/Protect, suppression)	3	Succès
`address-list.spec.ts`	`AddressList` (sélection, ajout, suppression)	3	Succès
`app.spec.ts`	`App` (initialisation et bascule d'onglets)	2	Succès
Total de la suite	**Ensemble des fonctionnalités du lab 3**	**31**	**31 / 31 (100% Réussi)**

4. Étapes Nécessaires pour Installer et Exécuter le Projet

Pour reproduire l'environnement, installer les dépendances et exécuter l'application localement, suivez les commandes ci-dessous :

```
# 1. Se positionner dans le dossier du projet
cd lab3-shopping-list

# 2. Installer l'ensemble des dépendances nécessaires
npm install

# 3. Lancer le serveur de développement Angular
npm start
# L'application est alors accessible dans le navigateur à : http://localhost:4200

# 4. Lancer la suite de 31 tests unitaires automatisés
npm test

# 5. Compiler l'application pour le déploiement de production
npm run build
```

5. Conclusion

Toutes les exigences du laboratoire 3 ont été satisfaites avec succès : respect strict du wireframe de la liste de courses, découpage en composants réutilisables, communication par `@Input` / `@Output`, gestion d'état asynchrone par Observables RxJS, et couverture complète par 31 tests unitaires.